INFORME TÉCNICO, DEFINICIÓN DEL CASO Y GESTIÓN DE PROYECTO

Asignatura: *Desarrollo de Aplicaciones Móviles (DSY1105)*
Integrantes: *Fabián Oyarce - Benjamín Salas*
Proyecto: *Sistema Móvil de Registro y Gestión de Ventas/Convenios*
Tecnologías: *Android, Kotlin 2.0, Jetpack Compose, Material Design 3, Room DB, MVVM*
Versión del Entregable: *Completado y Sincronizado en GitHub*

1. Definición del Caso y Requerimientos de Negocio

1.1 Contexto de Negocio

La aplicación móvil responde a la necesidad de automatizar y digitalizar la toma de registros de ventas y convenios en terreno. El sistema debe operar con alta confiabilidad en entornos con conectividad limitada o nula (modo *offline-first*), garantizando la integridad de la información mediante persistencia en base de datos local y validaciones estrictas en el punto de captura.

1.2 Requerimientos Funcionales (RF)

- **RF-01 (Registro de Ventas):** *Permitir el ingreso de nuevas ventas asociando tipo de servicio, cliente ficticio, fecha de operación, monto, código de autorización y observaciones.*
- **RF-02** *(Validación de Negocio): Impedir el guardado de datos incompletos, validar que el monto sea un número estrictamente mayor a 0 y asegurar la unicidad del código de autorización para evitar duplicidades.*
- **RF-03** *(Consulta e Histórico): Desplegar una lista reactiva en tiempo real con todas las ventas registradas localmente en orden cronológico inverso.*
- **RF-04** *(Edición y Actualización): Permitir la selección de un registro del histórico para cargar sus antecedentes en el formulario y modificar sus campos.*
- **RF-05** *(Recursos Nativos del Sistema): Incorporar selección de fechas mediante un diálogo nativo del sistema operativo (DatePickerDialog) y captura visual de comprobantes mediante la cámara del dispositivo.*

1.3 Requerimientos No Funcionales (RNF)

- **RNF-01** *(Arquitectura): Implementar la arquitectura recomendada por Google: MVVM (Model-View-ViewModel) con patrón Repository.*
- **RNF-02** *(Persistencia): Uso exclusivo de Room Database (v2.6.1) con Corrutinas y StateFlow para reactividad.*
- **RNF-03** *(Interfaz de Usuario): Diseño moderno mediante Jetpack Compose y Material Design 3, incorporando íconos descriptivos en cada campo de entrada.*
- **RNF-04** *(Control de Versiones): Control de cambios profesional en Git/GitHub con commits atómicos descriptivos y gestión de conflictos.*
-

2. Informe Técnico y Arquitectura de Software

2.1 Estructura Arquitectónica (MVVM)

La solución sigue un flujo unidireccional de datos (*Unidirectional Data Flow - UDF*), dividida en tres capas principales:

[Capa de Presentación] [Capa de Negocio] [Capa de Datos]

(Jetpack Compose UI) <=======> (VentaViewModel) <=======> (VentaRepository)

- ListaVentasScreen - VentaUiState - VentaDao

- VentaFormScreen - Coroutines - Room AppDatabase

- **Capa de Datos:** *Compuesta por AppDatabase, VentaDao y VentaRepository. Es responsable de ejecutar las consultas SQL asíncronas y exponer las entidades mediante Flow<List<VentaEntity>>.*

- **Capa de Negocio** *(ViewModel): VentaViewModel mantiene el estado de la UI en un MutableStateFlow<VentaUiState>. Procesa la lógica de validación, verifica duplicados en Room y maneja la carga de registros para su edición.*

- **Capa de Presentación (UI):** *Pantallas construidas en declarativo con Jetpack Compose. Reaccionan automáticamente a las emisiones de estado del ViewModel.*

2.2 Modelo de Datos (Room Schema)

Tabla Entity: *ventas*

Campo	Tipo SQL / Kotlin	Restricción / Clave	Descripción del Campo
id	INTEGER / Int	PRIMARY KEY (autoGenerate = true)	Identificador único del registro.
tipoServicio	TEXT / String	NOT NULL	Tipo de servicio o convenio.
fechaOperacion	TEXT / String	NOT NULL	Fecha formateada (DD/MM/AAAA).
cliente	TEXT / String	NOT NULL	Nombre o razón social ficticia del cliente.
monto	REAL / Double	NOT NULL	Valor numérico positivo de la venta.
codigoAutorizacion	TEXT / String	UNIQUE, NOT NULL	Código único de validación de la transacción.
respaldoUri	TEXT / String?	NULLABLE	Ruta o URI de la fotografía capturada.
estado	TEXT / String	NOT NULL ("Pendiente")	Estado del registro.
observaciones	TEXT / String	NOT NULL	Notas o comentarios adicionales.

3. Informe de Gestión de Proyecto y Control de Versiones

3.1 Estrategia de Versionado Git

Se estableció la rama main como rama principal de producción sincronizada con el repositorio remoto de GitHub (FabianDuoc2026/DESARROLLO-DE-APLICACIONES-MOVILES-EV2).

- **Resolución de Conflictos: Se superó la divergencla de historiales y archivos no sincronizados mediante rebase e integración forzada limpia (git push -f origin main), asegurando que ambos integrantes trabajen sobre un árbol de commits sin inconsistencias.**
- **Convención de Commits: Implementación de *Conventional Commits* (feat:, docs:, fix:) para mantener trazabilidad.**

3.2 Plan de Trabajo y Reparto de Responsabilidades

MATRIZ DE TRABAJO EN EQUIPO

Benjamín Salas *(Completado y Sincronizado en Main)*

• Configuración de Room DB (Entity, DAO, Database).

• Implementación de VentaRepository y VentaViewModel.

• Creación de VentaFormScreen con validaciones avanzadas.

• Integración de DatePickerDialog (Recurso Nativo 1).

• Íconos Material Design 3 en inputs y flujo de edición.

▼ (Sincronización Git Pull)

Fabián Oyarce *(Listo para Inyección estructura de proyecto)*

• Inyección de Cámara (Recurso Nativo 2) en VentaFormScreen.

• Captura de foto mediante rememberLauncherForActivityResult.

• Vinculación del URI resultante con viewModel.onRespaldo...

4. Matriz de Cumplimiento de la Rúbrica

Criterio de Rúbrica	Ponderación	Implementación en la Aplicación
IE 2.1.1: Persistencia de Datos (Room)	**20%**	**Implementación completa de Room (Entity, DAO, Database) con operaciones CRUD, actualización de registros y consultas asíncronas con Corrutinas.**
IE 2.1.2: Interfaz de Usuario y Material 3	**15%**	**Pantallas en Jetpack Compose, componentes M3 (OutlinedTextField, Card, IconButton), e íconos explicativos contextuales en cada campo.**
IE 2.2.1: Lógica de Negocio y Validaciones	**20%**	**Validaciones en tiempo real en el ViewModel: prevención de campos vacíos, montos numéricos válidos (>0) y control de unicidad de código.**
IE 2.3.1: Arquitectura de Software (MVVM)	**15%**	**Separación estricta de capas UI, ViewModel y Repository, manteniendo el estado encapsulado en StateFlow.**

IE 2.4.1: Recursos Nativos del Dispositivo	**15%**	**Integración de 2 recursos nativos: DatePickerDialog del sistema operativo para fechas y preparación para la Cámara en captura de respaldo.**
IE 2.5.1: Control de Versiones (Git)	**15%**	**Historial limpio en GitHub, resolución exitosa de conflictos, estructura de carpetas estándar y trazabilidad de commits.**